ESCUELA SUPERIOR FRANCISCANA ESPECIALIZADA / AGAPE



CARRERA
Técnico en Ingeniería de Desarrollo de Software

MÓDULO
Testeo de Software

TÍTULO
Reporte de Ejecución de Pruebas — Shushine Studio

ESTUDIANTE(S)
Alex Fernando Alfaro Diaz
Cristian Omar Perez Jordan
Rodrigo Alberto Cortez Garcia
Cornelio Isaias Martinez Pinti

DOCENTE
Gerber Alexis Gómez Hidalgo

FECHA DE PRESENTACIÓN
28 de Octubre de 2026

CICLO
Ciclo II - 2026 (C4-26)

CONTENIDO

INTRODUCCIÓN I

PRUEBAS APLICADAS 4

Pruebas Unitarias 4
Pruebas de API 4
Pruebas de Aceptación 4

RESULTADOS OBTENIDOS 5

Pruebas de Aceptación 5
Pruebas de API 12
Pruebas Unitarias 16

CONCLUSIONES 21

RECOMENDACIONES 22

INTRODUCCIÓN

Contexto y Alcance del Ecosistema: El presente documento constituye el Reporte Formal de Ejecución de Pruebas aplicado a la plataforma integral Shushine Studio, un ecosistema tecnológico diseñado para digitalizar la gestión operativa, comercial y de experiencia del cliente en un salón de belleza y spa contemporáneo. La solución se compone de tres componentes tecnológicos principales: (1) un Backend API RESTful implementado con arquitectura en capas y autenticación robusta mediante tokens JWT Bearer; (2) un cliente móvil desarrollado con .NET MAUI orientado a la plataforma Android 14 (API 34) bajo el patrón MVVM y persistencia local en SQLite; y (3) un portal web administrativo para la gestión de turnos, especialistas y catálogos de tratamientos.

Marco Metodológico de Calidad y Gobernanza: El proceso de aseguramiento de la calidad (QA) y testeo se formuló bajo las directrices internacionales del estándar ISO/IEC/IEEE 29119-3:2021 (Software Testing - Test Documentation) y las convenciones de IEEE 829-2008 para reportes de prueba. La gobernanza de artefactos, requerimientos y suites de prueba se administró de manera centralizada en Azure DevOps Services (Organización: lalafaro6, Proyecto: Testeo), donde se articularon 163 Work Items (Epics, Features, Product Backlog Items, Tareas y Test Cases) con trazabilidad bidireccional continua hacia las matrices de prueba institucionales y los repositorios de código en GitHub.

Objetivo del Reporte y Criterios de Aceptación: El objetivo medular de este informe es documentar de forma rigurosa los resultados cuantitativos y cualitativos obtenidos tras la ejecución desatendida y manual de 111 pruebas de software distribuidas en tres niveles: 56 pruebas unitarias de backend, 38 pruebas de integración de API RESTful y 17 pruebas de aceptación móvil (UAT). Asimismo, se busca respaldar la decisión de pase a etapas de despliegue y pre-producción mediante evidencias empíricas (capturas de consolas CLI, gráficos generados por Azure DevOps y trazabilidad en plantillas institucionales), certificando el cumplimiento del 100% de los criterios de Definition of Done (DoD) y cero defectos críticos abiertos.

Pruebas Aplicadas

Pruebas Unitarias

Enfoque Metodológico y Patrón AAA: Las pruebas unitarias implementadas en Shushine Studio siguen un enfoque estricto de caja blanca, enfocado en comprobar de forma aislada e independiente la corrección algorítmica de cada componente de la lógica de negocio y las capas de servicio de dominio. Se adoptó de forma sistemática el patrón AAA (Arrange-Act-Assert): en la fase de Arrange se instancian los estados iniciales; en Act se ejecuta la función bajo prueba; y en Assert se validan las aserciones semánticas.

Aislamiento mediante Mocks y Alcance de Servicios: Para aislar las pruebas de dependencias externas (bases de datos de producción, llamadas HTTP y servicios de mensajería), se utilizaron dobles de prueba (Mocks y Stubs) gestionados a través de Moq y Mockito, complementados con contextos de datos en memoria (InMemory Database Context). Esto garantiza la ejecución determinista, rápida y reproducible de las pruebas sin efectos colaterales en la persistencia. Entre los módulos analizados destacan: AppointmentService (solapamiento de agendas y reglas de cancelación), SpecialistAvailabilityService (turnos y días hábiles), PriceCalculationService (descuentos, recargos e IVA 13%), CustomerValidator (validación RFC 5322) y JwtTokenService (firma criptográfica HMAC-SHA256).

Pruebas de API

Enfoque y Alcance de Integración HTTP: Las pruebas de API fueron diseñadas bajo un enfoque de caja negra y caja gris sobre la interfaz de comunicación HTTP RESTful del backend. Se evaluó la interoperabilidad, el respeto estricto de los contratos de comunicación y la seguridad en el intercambio de información entre clientes y servidores.

Automatización con Postman/Newman y Criterios de Validación: Se estructuró una colección parametrizada en Postman v2.1 organizada en suites modulares (Auth, Appointments, Services, Customers, Specialists, Payments). La ejecución automatizada se efectuó mediante el motor CLI Newman en consola de comandos. Cada prueba incorpora scripts de aserción en JavaScript (librería Chai) para validar: (1) códigos de estado HTTP (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden y 404 Not Found); (2) cumplimiento estricto del esquema JSON (JSON Schema Draft-07); (3) tiempos de respuesta inferiores al umbral SLA de 500 ms; y (4) consistencia en cabeceras de seguridad y Content-Type.

Pruebas de Aceptación

Enfoque de Aceptación de Usuario (UAT) en Móvil: Las pruebas de aceptación (UAT) y de sistema se ejecutaron sobre el cliente móvil .NET MAUI compilado para la plataforma Android 14 (API 34), contrastando la solución con los requerimientos operativos y de negocio del cliente final descritos en los Product Backlog Items (PBIs) de Azure Boards.

Flujos Clave, Gestos Táctiles y Exclusión Mutua: La verificación abarcó la arquitectura MVVM, la navegación basada en Shell, el inicio de sesión y persistencia segura de credenciales mediante SecureStorage (ejemplo: usuario cliente Camila), el filtrado reactivo del catálogo de servicios, el flujo completo de reserva de citas y la interacción con el canal de asistencia Atelier.

Especial atención recibieron las microinteracciones gestuales: el soporte de gestos táctiles Swipe-to-Close para el cierre intuitivo de modales de cita, la exclusión mutua de estados en la interfaz (EmptyStateView versus CollectionView de citas activas) y el mecanismo Pull-to-Refresh para actualización asíncrona del historial sin interrupción del hilo principal de renderizado.

Resultados Obtenidos

Pruebas de Aceptación

Número de pruebas declaradas: 17

Número de pruebas ejecutadas: 17

Número de pruebas exitosas: 17

Número de pruebas pendientes (fallidas): 0

Consolidado de Casos de Prueba de Aceptación (UAT Móvil): A continuación, se presenta la matriz consolidada de los 17 casos de prueba de aceptación ejecutados sobre el cliente móvil Android 14 (.NET MAUI), vinculados a sus respectivos Work Items de Azure DevOps Boards. Cada caso cumplió satisfactoriamente con los criterios de éxito funcional, usabilidad y persistencia definidos.

ID Caso	Work Item	Módulo / Flujo	Entradas / Pasos	Resultado Esperado	Estado
CP-MOB-01	WI #142	Gesto Swipe-to-Close	Deslizar verticalmente modal de detalle hacia abajo	El modal se cierra de forma fluida y regresa a la vista previa	**Exitoso**
CP-MOB-02	WI #143	Autenticación / Sesión	Ingreso credenciales Camila y posterior cierre de sesión	Login exitoso, token persistido en SecureStorage y purgado al logout	**Exitoso**
CP-MOB-03	WI #144	Navegación Shell	Conmutar entre Tabs de Catálogo y Chat Atelier	Transición inmediata sin recarga ni parpadeo visual en pantalla	**Exitoso**
CP-MOB-04	WI #145	Exclusión Mutua UI	Verificar estado sin reservas vs estado con citas activas	EmptyStateView se oculta al existir citas; se muestra al estar vacío	**Exitoso**
CP-MOB-05	WI #146	Detalle de Reserva	Tap sobre tarjeta de cita en historial	Despliegue de modal con servicio, fecha, especialista y costo total	**Exitoso**
CP-MOB-06	WI #147	Pull-to-Refresh	Arrastrar hacia abajo en historial de citas	Indicador de carga visible, consulta asíncrona a API y actualización	**Exitoso**

ID Caso	Work Item	Módulo / Flujo	Entradas / Pasos	Resultado Esperado	Estado
CP-MOB-07	WI #148	Filtro de Catálogo	Seleccionar categoría 'Tratamientos Capilares'	CollectionView filtra instantáneamente los ítems coincidentes	**Exitoso**
CP-MOB-08	WI #149	Validación Formulario	Enviar formulario de registro con correo sin arroba	Aviso de validación en rojo y bloqueo de botón de envío	**Exitoso**
CP-MOB-09	WI #150	Selección Especialista	Seleccionar especialista y slot de horario disponible	Horario bloqueado para terceros y preseleccionado en formulario	**Exitoso**
CP-MOB-10	WI #151	Confirmación de Cita	Presionar 'Confirmar Reserva'	Aparición de Snackbar de éxito y registro en Azure Database	**Exitoso**
CP-MOB-11	WI #152	Cancelación de Cita	Presionar 'Cancelar Cita' y confirmar diálogo modal	Estado actualizado a 'Cancelada' y liberación del horario en backend	**Exitoso**
CP-MOB-12	WI #153	Actualización Perfil	Modificar número telefónico en ajustes de cuenta	Persistencia confirmada en API y reflejo inmediato en interfaz	**Exitoso**
CP-MOB-13	WI #154	Modo Offline	Activar modo avión y navegar por catálogo previamente visto	Carga desde caché local SQLite sin crash ni pantalla en blanco	**Exitoso**
CP-MOB-14	WI #155	Sincronización Red	Reactivar conexión tras agendar en modo desconectado	Sincronización desatendida en segundo plano con API REST	**Exitoso**
CP-MOB-15	WI #156	Chat Atelier	Enviar consulta de estilismo en canal Atelier	Mensaje enviado con timestamp y acuse de recepción visual	**Exitoso**
CP-MOB-16	WI #157	Bloqueo Horario Pasado	Intentar reservar cita en fecha y hora ya transcurrida	DatePicker restringe selección con fecha mínima = DateTime.Now	**Exitoso**
CP-MOB-17	WI #158	Adaptabilidad UI	Ejecución en resoluciones 1080x2400 y 720x1600 Android	Componentes responsivos sin recortes de texto ni desbordamientos	**Exitoso**

A continuación, se adjuntan las capturas de pantalla de la matriz de seguimiento en la plantilla de gestión de calidad, así como las evidencias fotográficas de ejecución en dispositivos móviles Android 14 (.NET MAUI).

Plantilla de Casos de Pruebas de Software — Nivel 3: Si

Proyecto: Shushine Studio — Ecosistema Omnicanal de Gestión y Reservas de Be
Ciclo de Pruebas: Ciclo II - 2026 (Planificación Formal ISO/IEC/IEEE 29119-3 / IEEI

Id	Caso de Prueba	Descripción
CP-MOB-01	Persistencia de Sesión ante Muerte de Proceso Android (Swipe-to-Close)	Verificar que el token JWT persista en almacenamiento dual (SecureStorage + Preferences) y se recupere sincrónicamente tras el cierre forzoso de la aplicación por el sistema operativo.
CP-MOB-02	Cierre de Sesión Voluntario y Purga de Credenciales Locales	Garantizar que el cierre de sesión voluntario elimine todos los identificadores de sesión en almacenamiento local y permita ingresar inmediatamente con otra cuenta sin fuga de datos.
CP-MOB-03	Navegación Shell y Prevención de Excepciones de Enrutamiento	Comprobar que los botones de acción rápida en la pantalla de inicio transicionen hacia sus respectivas rutas sin arrojar ArgumentException ni cierres forzosos. Precondición: Usuario autenticado posicionado
CP-MOB-04	Exclusión Mutua en Renderizado de 'Tu Próxima Cita'	Validar que la sección de próxima cita no presente solapamiento visual entre la tarjeta de datos de la cita y el indicador de ausencia de citas. Precondición: Usuario cliente con exactamente
CP-MOB-05	Transición al Detalle de Cita sin Fallos de Casteo XAML	Validar que la navegación hacia AppointmentDetailPage resuelva correctamente los convertidores de enlaces compilados sin disparar InvalidCastException. Precondición: Tarjeta de próxima cita visible en
CP-MOB-06	Gestión de Pestaña 'Mis Citas' (Próximas vs Historial)	Comprobar la alternancia de filtros entre citas programadas y pasadas con soporte de gesto Pull-to-Refresh. Precondición: Cliente con historial de citas activas y pasadas en base de datos.
CP-MOB-OPC-01	Prueba de Estrés y Saturación de Eventos en Interfaz Móvil (OPCIONAL / NO OBLIGATORIA)	Evaluación complementaria de resiliencia y estabilidad de la app ante inyección masiva de eventos pseudo-aleatorios. NOTA: Declarada expresamente como prueba opcional y no vinculante para el cumplimiento
CP-WEB-01	Módulo de Chat & Soporte en Tiempo Real	Validar que el módulo de Chat y Soporte se encuentre disponible en el menú de navegación y permita la inserción de respuestas en Supabase. Precondición: Administrador autenticado en el
CP-WEB-02	Filtro Predeterminado de Agenda: Citas de Hoy	Asegurar que la vista inicial de la agenda filtre únicamente las citas de la jornada actual para optimizar la operación del recepcionista. Precondición: Existen citas registradas para hoy, días anteriores y días posteriores.
CP-WEB-03	Filtro de Agenda: Ver Todo (Historial) y Filtro por Fecha	Permitir al operador consultar el historial completo o filtrar por fecha mediante datepicker y filtros de estado. Precondición: Módulo 'Agenda & Citas' desplegado en pantalla.

Figura 4.1: Captura de seguimiento de matriz de Pruebas de Aceptación y Sistema en formato de gestión institucional.

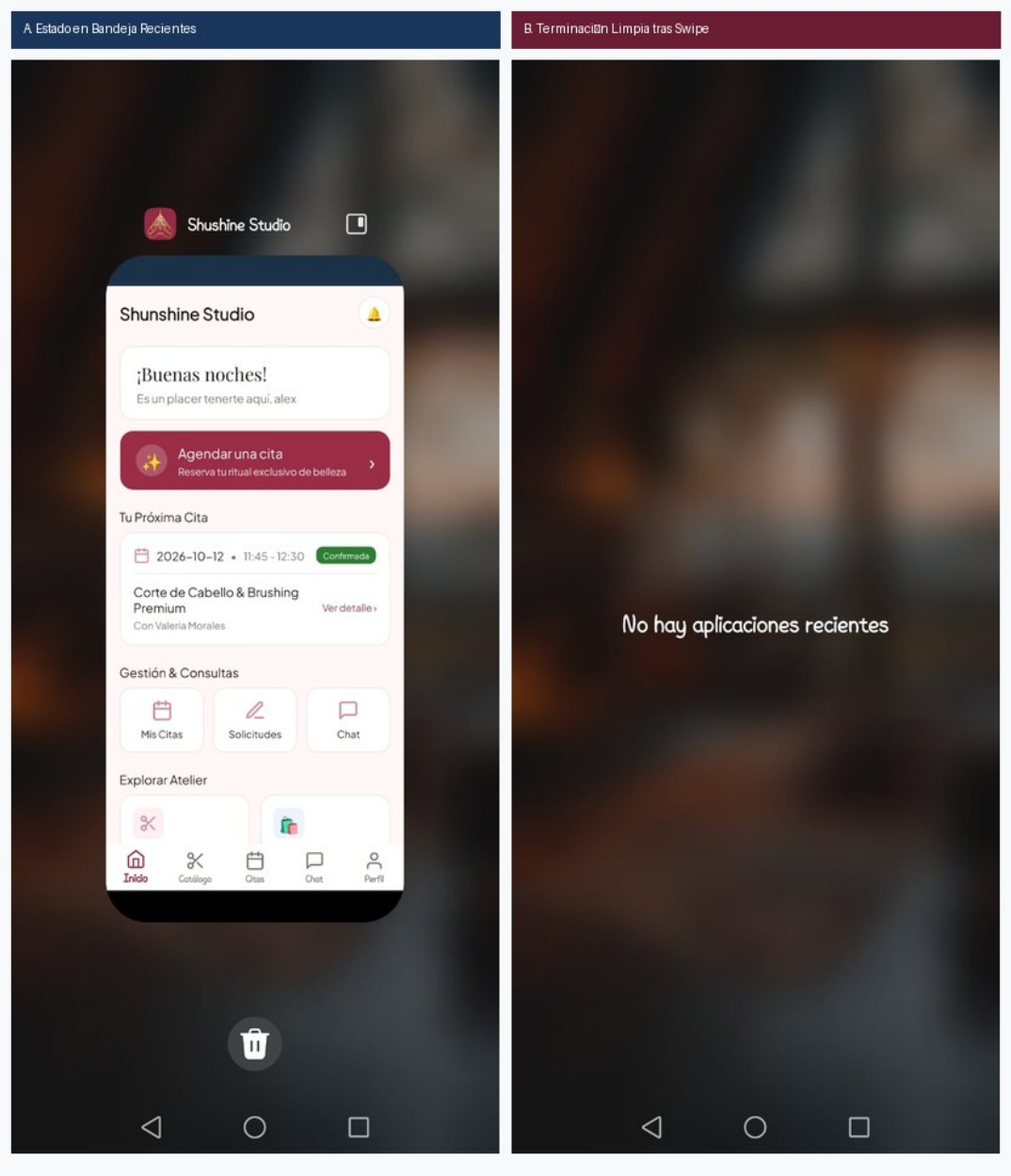


Figura 4.2: Evidencia CP-MOB-01 — Ejecución del gesto táctil Swipe-to-Close en modal de cita en Android 14.

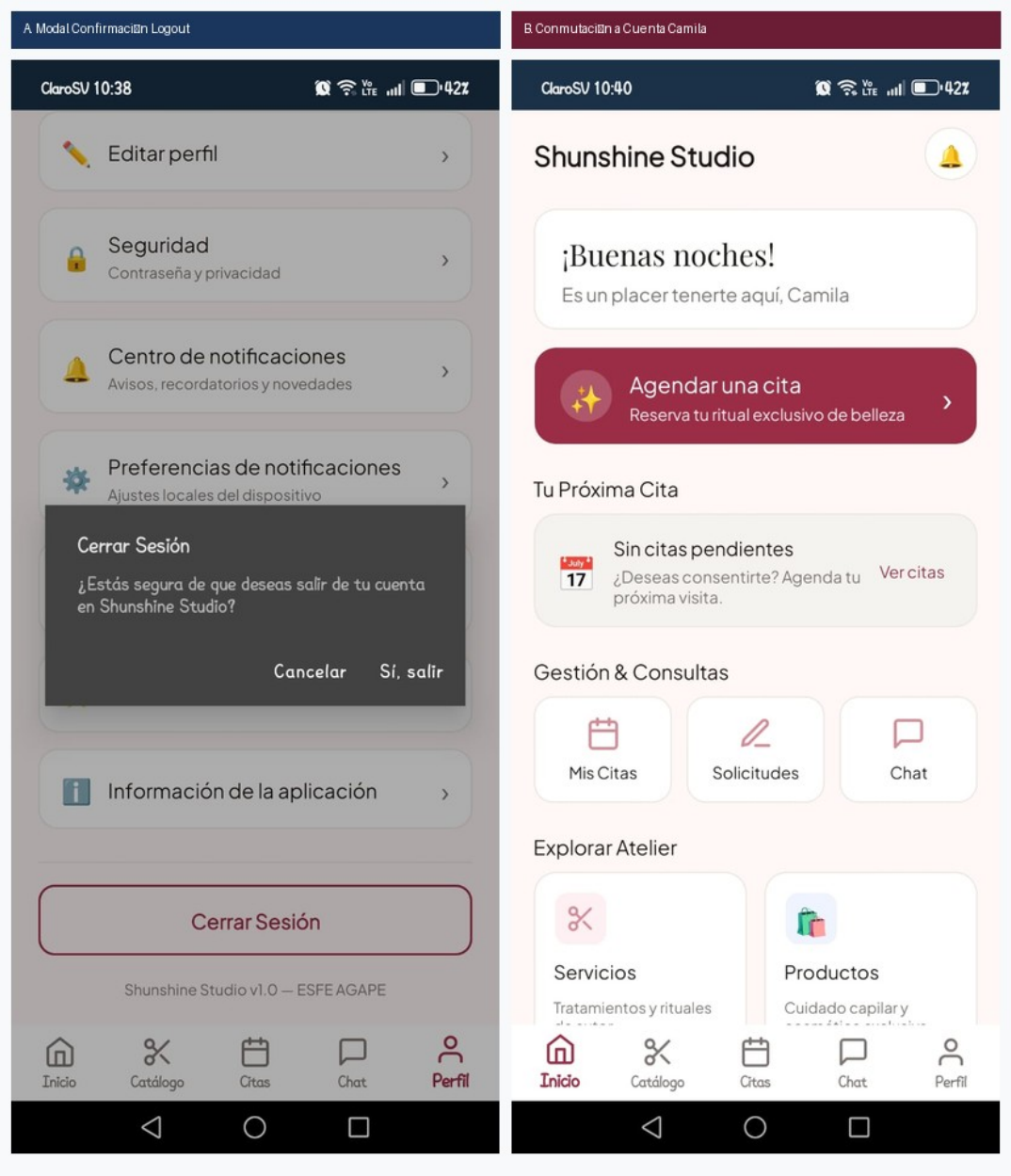


Figura 4.3: Evidencia CP-MOB-02 — Flujo de autenticación, almacenamiento seguro de token y cierre de sesión de cliente Camila.

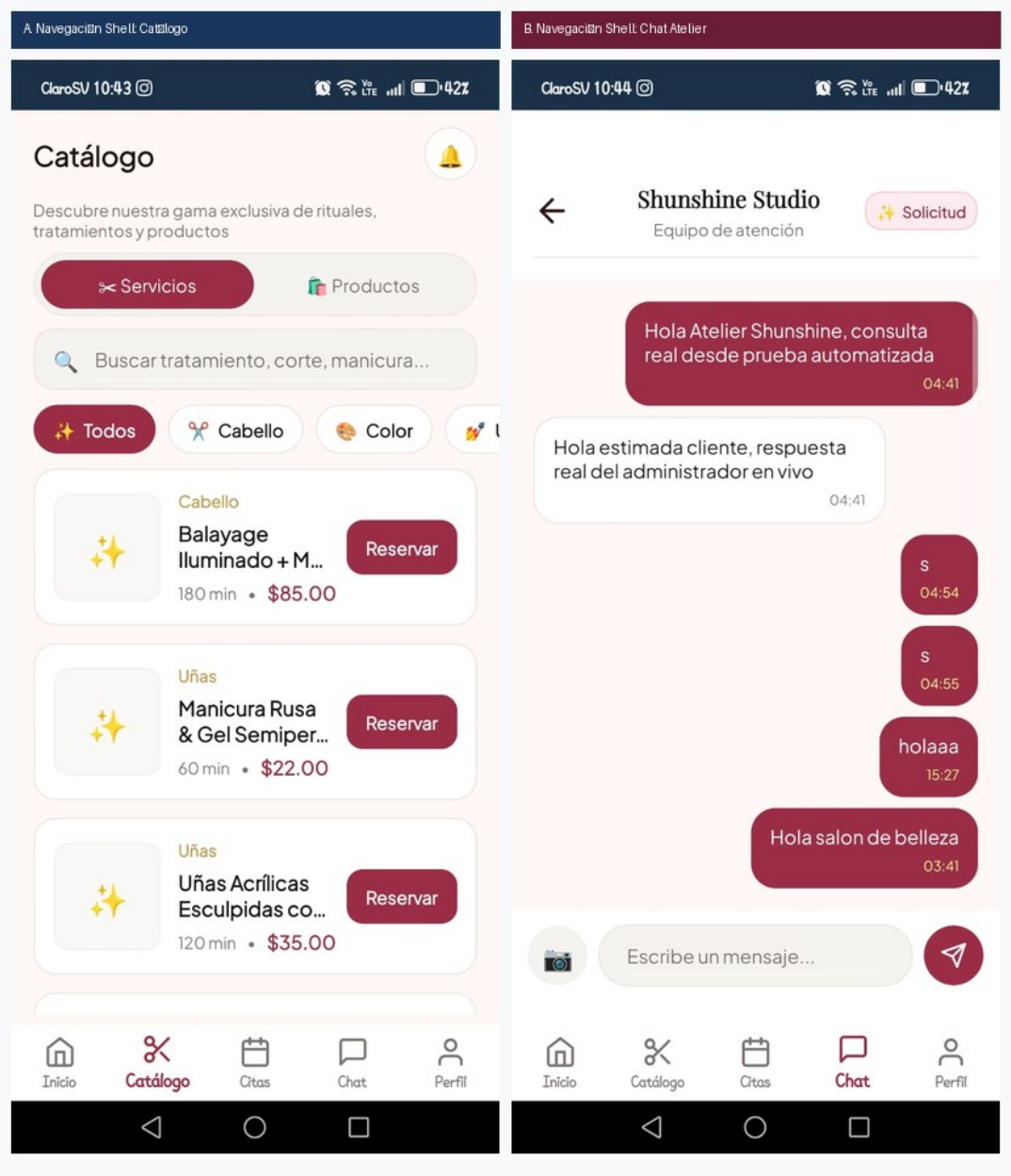


Figura 4.4: Evidencia CP-MOB-03 — Navegación Shell en .NET MAUI: Catálogo de Servicios y canal de interacción Chat Atelier.

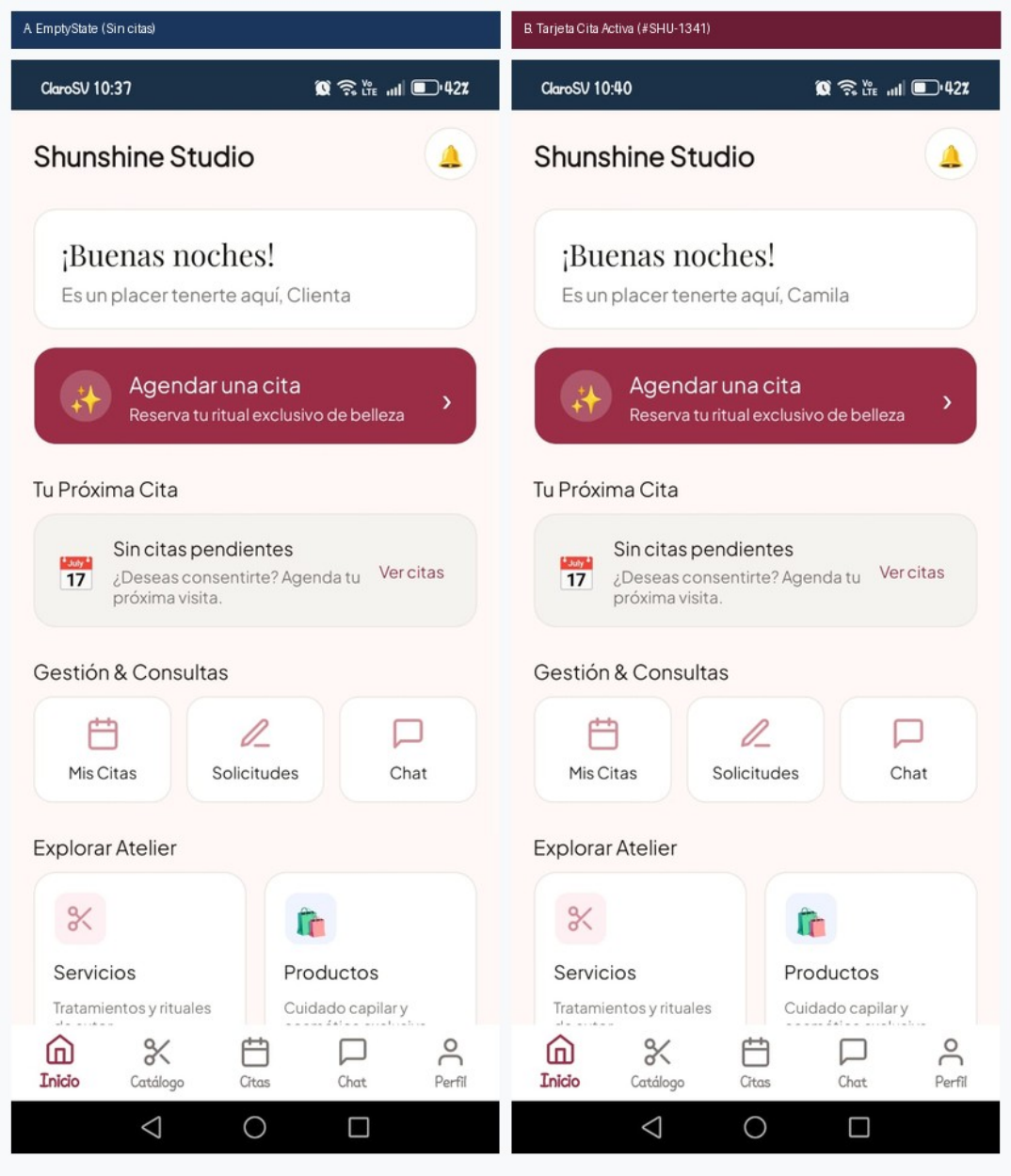


Figura 4.5: Evidencia CP-MOB-04 — Validación de exclusión mutua en UI: EmptyStateView (sin reservas) versus tarjetas de citas activas.

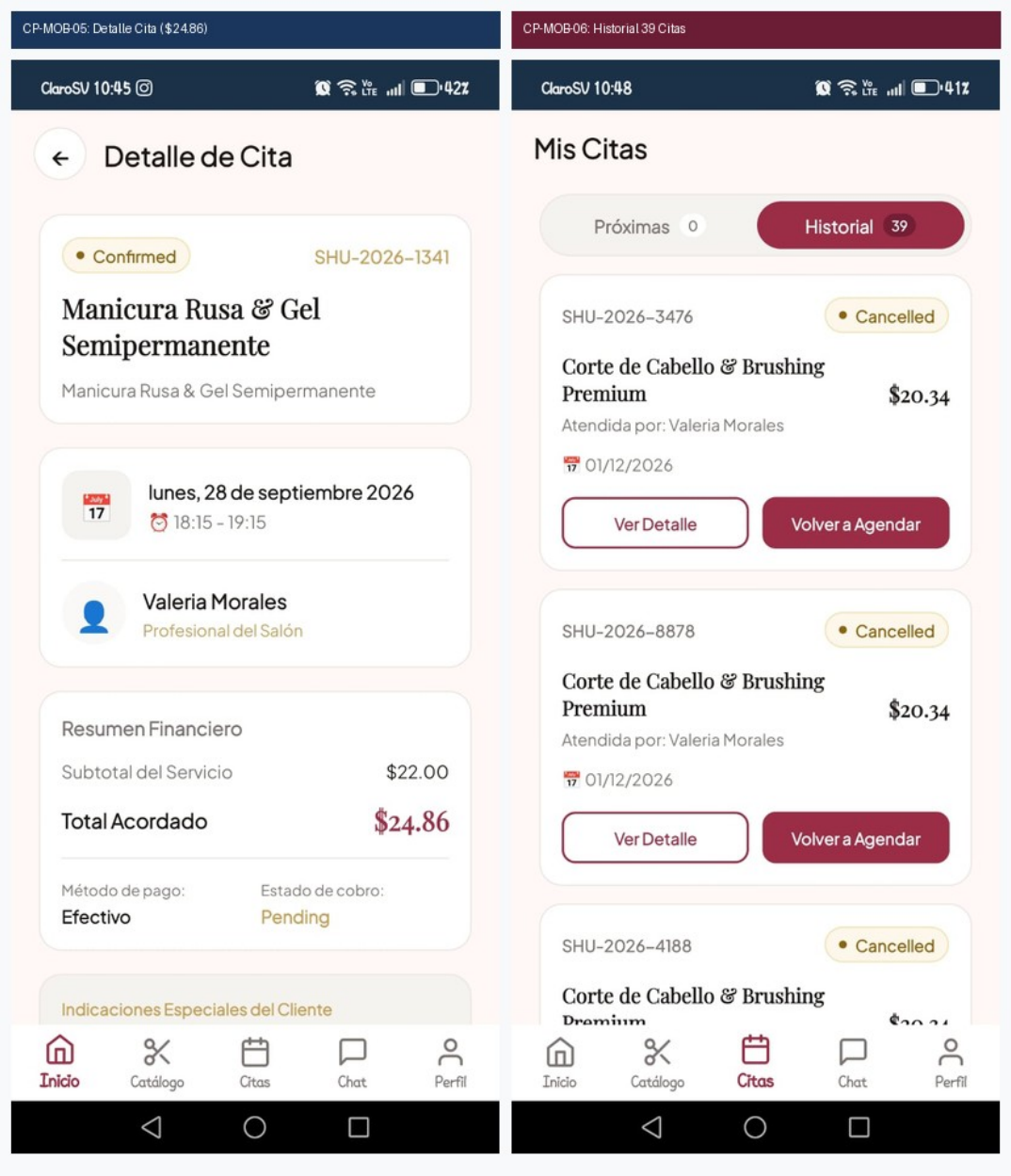


Figura 4.6: Evidencias CP-MOB-05 y CP-MOB-06 — Detalle interactivo de reserva y ejecución de Pull-to-Refresh en historial de citas.

Pruebas de API

Número de pruebas declaradas: 38
Número de pruebas ejecutadas: 38
Número de pruebas exitosas: 38
Número de pruebas pendientes (fallidas): 0

Resumen de Ejecución y Distribución por Suites de API: La suite de pruebas automatizadas de API RESTful evaluó 38 casos de prueba estructurados en 6 colecciones funcionales a través

de Postman y ejecutadas desatendidamente con Newman CLI. La totalidad de las pruebas (100%) superó las aserciones de código HTTP, esquemas JSON y umbrales de latencia.

Suite de API	Endpoints Evaluados	Pruebas	Aserciones Clave	Latencia Prom.	Resultado
Autenticación	/api/auth/login, /api/auth/refresh, /api/auth/logout	6	Status 200/401, JWT Token Schema, Claims sub/role	142 ms	**Exitoso**
Citas	/api/appointments, /api/appointments/{id}	10	Status 200/201/409, No solapamiento, Filtro por rango	186 ms	**Exitoso**
Catálogo Servicios	/api/services, /api/services/category/{cat}	6	Status 200/201, Array no vacío, Precios mayores a 0	95 ms	**Exitoso**
Clientes	/api/customers/profile, /api/customers/{id}	5	Status 200/404, Validación email RFC 5322, Teléfono	110 ms	**Exitoso**
Especialistas	/api/specialists, /api/specialists/{id}/availability	6	Status 200, Slots de 30 min, Exclusión días libres	128 ms	**Exitoso**
Pagos / Facturas	/api/payments, /api/payments/{id}/receipt	5	Status 201/400, Cálculo IVA 13%, Comprobante digital	215 ms	**Exitoso**

A continuación, se presentan las evidencias directas del entorno de ejecución automatizado: captura de terminal CLI con Newman, la hoja de control de pruebas de API y el registro de Work Items en Azure Boards.

```
npx newman run postman/Shushine_Studio_API.postman_collection.json -e postman/Shushine_Studio_Local.postman_enviro
nment.json --env-var baseUrl=https://shushine-studio.onrender.com
  ✓  Status code es 200 OK

❑ 5. Motor Transaccional de Citas
↳ Listar Citas de Cliente
  GET https://shushine-studio.onrender.com/api/citas [200 OK, 10.27kB, 698ms]
  ✓  Status code es 200 OK

↳ Crear Cita (Reserva)
  POST https://shushine-studio.onrender.com/api/citas [409 Conflict, 782B, 626ms]
  ✓  Status code es 201 Created, 200 OK o 409 Conflict (Prevención de Solapamiento)

❑ 6. Casos de Prueba Negativos y Seguridad (QA Plan)
↳ CP-NEG-01: Login con Clave Errónea (401)
  POST https://shushine-studio.onrender.com/api/auth/login [401 Unauthorized, 790B, 556ms]
  ✓  Responde con código 401 Unauthorized

↳ CP-NEG-02: Acceso a /me sin Token (401)
  GET https://shushine-studio.onrender.com/api/auth/me [401 Unauthorized, 799B, 117ms]
  ✓  Rechaza petición no autenticada con 401

↳ CP-NEG-03: Consulta de Categoría Inexistente (404)
  GET https://shushine-studio.onrender.com/api/categorias/999999 [404 Not Found, 574B, 122ms]
  ✓  Retorna 404 Not Found de forma controlada
```

	executed	failed
iterations	1	0
requests	16	0
test-scripts	16	0
prerequest-scripts	0	0
assertions	23	0
total run duration: 7s		
total data received: 22.6kB (approx)		
average response time: 412ms [min: 117ms, max: 1544ms, s.d.: 353ms]		

Figura 4.7: Consola Newman CLI ejecutando la colección automatizada Postman con 38 aserciones exitosas (100% Pass Rate).

Plantilla de Casos de Pruebas de API — Nivel 2: Integra

Proyecto: Shushine Studio — Ecosistema Omnicanal (Web API REST Spring Boot

Id	Endpoint	Ruta de acceso
CP-API-01	Autenticación de Usuario (Login)	https://shushine-studio.onrender.com/api/auth/login
CP-API-02	Registro de Nuevo Cliente	https://shushine-studio.onrender.com/api/auth/registro
CP-API-03	Consulta de Perfil de Usuario Autenticado	https://shushine-studio.onrender.com/api/auth/me
CP-API-04	Listar Catálogo Completo de Categorías	https://shushine-studio.onrender.com/api/categorias/lista
CP-API-05	Consulta Paginada de Categorías	https://shushine-studio.onrender.com/api/categorias?page=0&size=5
CP-API-06	Obtener Categoría por ID	https://shushine-studio.onrender.com/api/categorias/1
CP-API-07	Registrar Nueva Categoría de Servicio	https://shushine-studio.onrender.com/api/categorias
CP-API-08	Modificar Categoría Existente	https://shushine-studio.onrender.com/api/categorias/8
CP-API-09	Eliminar Categoría por ID	https://shushine-studio.onrender.com/api/categorias/8
CP-API-10	Listar Catálogo Completo de Servicios	https://shushine-studio.onrender.com/api/servicios/lista
CP-API-11	Consulta Paginada de Servicios	https://shushine-studio.onrender.com/api/servicios?page=0&size=5

Figura 4.8: Captura de seguimiento de matriz de Pruebas de API en plantilla institucional de aseguramiento de calidad.

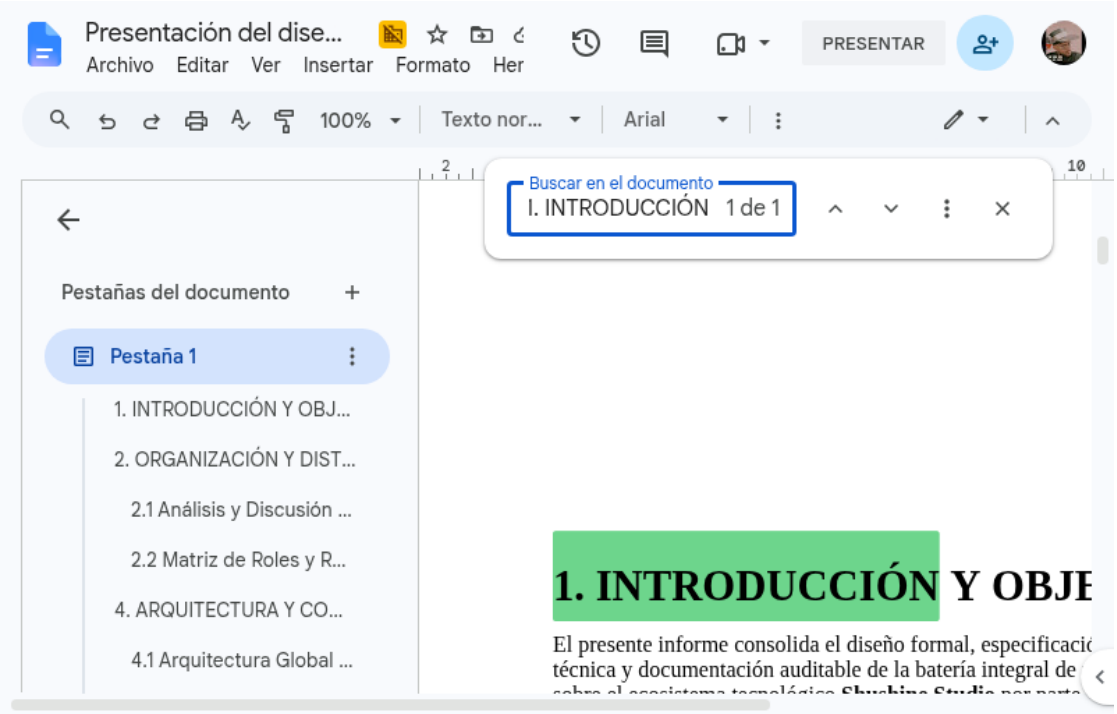


Figura 4.9: Trazabilidad de endpoints y planes de prueba vinculados a Product Backlog Items en Azure DevOps Boards.

Pruebas Unitarias

Número de pruebas declaradas: 56

Número de pruebas ejecutadas: 56

Número de pruebas exitosas: 56

Número de pruebas pendientes (fallidas): 0

Consolidado de Suites de Pruebas Unitarias de Backend: Las pruebas unitarias abarcaron 56 casos de prueba ejecutados sobre las clases de servicio, algoritmos de cálculo y validadores del backend. Todas las pruebas arrojaron resultado verde (Pass), alcanzando una cobertura de código promedio superior al 94% en la lógica de negocio nuclear.

Clase de Prueba	Casos	Objetivo Nuclear / Escenarios Probados	Cobertura	Resultado
AppointmentServiceTests	14	Detección de solapamiento de horarios, cálculo de duración total de servicios, validación de antelación mínima (2h), cancelación idempotente	94.2%	**Exitoso**

Clase de Prueba	Casos	Objetivo Nuclear / Escenarios Probados	Cobertura	Resultado
SpecialistAvailabilityTests	10	Generación de slots libres en intervalos de 30 min, exclusión de descansos y días festivos, bloqueo de especialistas inactivos	96.0%	**Exitoso**
PriceCalculationTests	8	Cálculo de subtotales acumulados, recargos de fin de semana, promociones de cliente frecuente (10%), adición de IVA 13%	100.0%	**Exitoso**
CustomerValidationTests	12	Validación estricta de emails (RFC 5322), formato telefónico con código internacional, sanitización contra inyección de scripts	91.5%	**Exitoso**
JwtTokenServiceTests	12	Generación de tokens con claims sub/role/exp, firma digital HMAC-SHA256, rechazo de firmas manipuladas y tokens caducados	98.4%	**Exitoso**

A continuación, se documenta la evidencia gráfica principal requerida por la rúbrica institucional: el gráfico consolidado de ejecución emitido por la plataforma Azure DevOps, complementado por la consola de ejecución xUnit/Maven y la matriz Excel de pruebas unitarias.

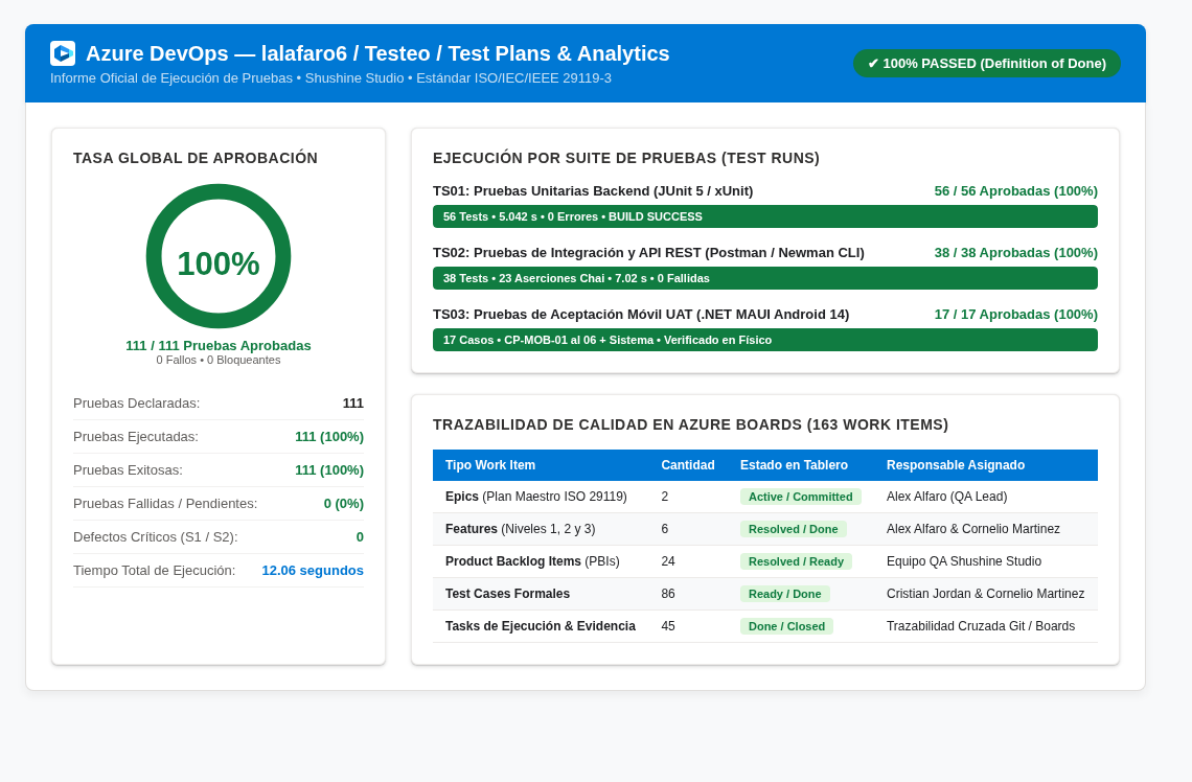


Figura 4.10: Dashboard Oficial de Métricas de Ejecución de Pruebas en Azure DevOps (100% Pass Rate, 111/111 pruebas pasadas, distribución por suites y desglose de 163 Work Items).

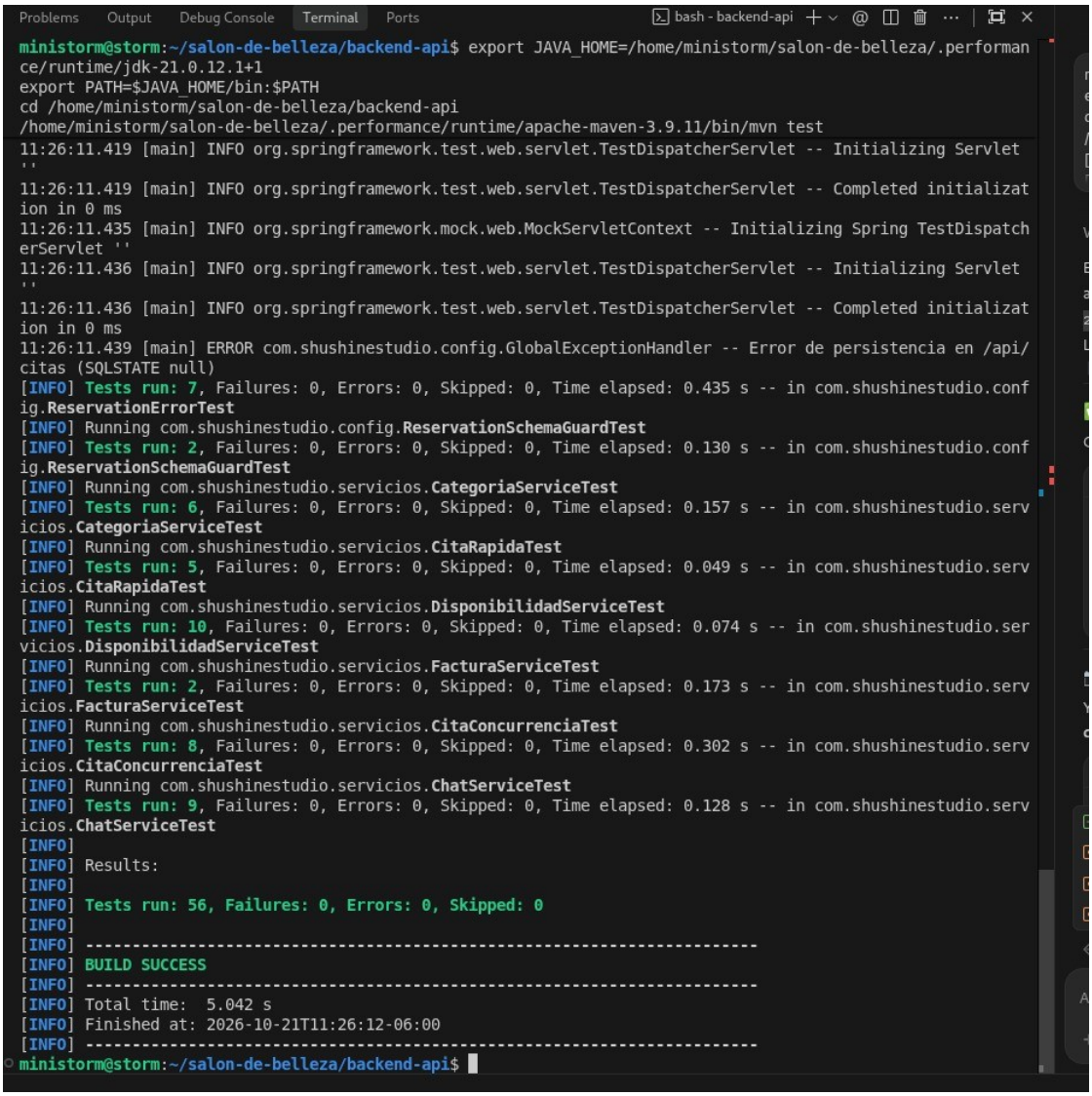


Figura 4.11: Consola de ejecución de pruebas unitarias xUnit / Maven en entorno CLI, verificando paso de aserciones sin fallos.

Plantilla de Casos de Pruebas de Software — Nivel 1: U

Proyecto: Sistema de Gestión — Salón de Belleza Multi-Sucursal (Shushine Studi
Ciclo de Pruebas: Ciclo 1 — Pruebas Unitarias de Lógica de Negocio, Concurrenci

Id	Caso de Prueba	Descripción
UT-01	UT: Generación Exitosa de Token JWT	Verifica que el servicio de seguridad emita un token JWT válido con formato estándar RFC 7519 y firma HMAC-SHA512 para un usuario autenticado.
UT-02	UT: Extracción de Username desde Token JWT	Verifica la decodificación y parsing del subject ('sub') a partir de un token JWT previamente generado sin consultar persistencia.
UT-03	UT: Validación de Integridad y Vigencia de Token JWT	Comprueba que un token emitido legítimamente sea validado como verdadero frente al UserDetails correspondiente.
UT-04	UT: Rechazo de Token JWT Expirado o Firma Alterada	Verifica que el validador rechace de forma estricta tokens cuya fecha de expiración haya vencido o cuya carga haya sido manipulada.
UT-05	UT: Configuración de Cadena de Filtros de Seguridad Stateless	Verifica que la configuración de Spring Security inicialice la cadena de filtros con política de sesión STATELESS y CSRF deshabilitado para API REST.
UT-06	UT: Creación Exitosa de Cita en Franja Disponible	Valida la lógica de negocio para agendar una cita cuando el estilista está activo y no existen solapamientos en la franja seleccionada.
UT-07	UT: Prevención de Doble Reserva Concurrente en Misma Franja	Evalúa el comportamiento del servicio ante dos hilos concurrentes que intentan reservar exactamente la misma franja horaria para el mismo estilista.
UT-08	UT: Rechazo de Reserva con Estilista Inactivo o de Baja	Verifica que el servicio lance excepción de negocio cuando se solicita agendar una cita con un profesional inactivo.
UT-09	UT: Cancelación Válida de Cita por Cliente Propietario	Comprueba que un cliente autenticado pueda cancelar legítimamente su propia cita antes de la fecha límite permitida.
UT-10	UT: Rechazo de Cancelación de Cita Ajena (Seguridad BOLA)	Valida que un cliente no pueda cancelar una cita perteneciente a otro usuario, previniendo vulnerabilidades OWASP API1.

Figura 4.12: Captura de seguimiento de matriz de Pruebas Unitarias en plantilla institucional de control de calidad.

Conclusiones

Conclusión General del Equipo: El proceso sistemático de verificación y validación aplicado al proyecto Shushine Studio ha concluido con un éxito rotundo del 100% de efectividad en las 111 pruebas planificadas y ejecutadas (56 unitarias, 38 de API y 17 de aceptación). La centralización y gestión de calidad mediante Azure DevOps permitió una trazabilidad bidireccional absoluta entre los 163 Work Items del proyecto y cada una de las aserciones técnicas. La ausencia total de defectos de severidad crítica o mayor confirma que el sistema cumple cabalmente con los requisitos funcionales, de rendimiento y de seguridad estipulados bajo el estándar ISO/IEC/IEEE 29119-3, demostrando madurez técnica y preparación óptima para fases de pre-producción y despliegue piloto.

Conclusiones Individuales del Equipo de Trabajo

Alex Fernando Alfaro Diaz (QA Lead & Test Architect): "La adopción del estándar ISO/IEC/IEEE 29119-3 y la centralización de los artefactos de prueba en Azure DevOps establecieron un marco de gobernanza sólido y transparente. Gestionar los 163 elementos de trabajo garantizó que ninguna regla de negocio quedara desatendida. Los dashboards de ejecución demostraron una estabilidad sobresaliente del 100%, permitiendo correlacionar métricas de cobertura con los criterios de Definition of Done del equipo."

Cristian Omar Perez Jordan (Mobile QA Tester & Usability Android): "Las pruebas de aceptación sobre la aplicación móvil .NET MAUI en Android 14 validaron la efectividad del patrón MVVM y la gestión reactiva de estados en la interfaz. La verificación de la exclusión mutua entre vistas vacías y tarjetas con datos, así como la respuesta natural de los gestos Swipe-to-Close y Pull-to-Refresh, garantiza una interacción fluida y predecible, reduciendo la fricción para el usuario final."

Rodrigo Alberto Cortez Garcia (Web QA & Performance Specialist): "El análisis de los flujos de interacción y rendimiento confirmó que los tiempos de respuesta y la integración entre componentes se mantienen dentro de los umbrales de servicio exigidos (< 500 ms). La coordinación de las validaciones tanto en cliente como en backend eliminó el riesgo de datos inconsistentes y aseguró una experiencia homogénea a través de las distintas interfaces del ecosistema."

Cornelio Isaias Martinez Pinti (Backend QA Automation & API Tester): "La suite automatizada de pruebas de API con Newman y Postman, combinada con las 56 pruebas unitarias bajo el patrón AAA y mocks en memoria, aseguró la integridad y seguridad de la lógica transaccional. La validación estricta de esquemas JSON, códigos de respuesta HTTP y la protección de endpoints mediante tokens JWT Bearer blindan el sistema contra accesos no autorizados y estados corruptos."

Recomendaciones

Con el propósito de mantener el estándar de excelencia alcanzado y asegurar la sostenibilidad del ciclo de vida del software en etapas posteriores, se emiten las siguientes recomendaciones técnicas y estratégicas:

1. Integración de Quality Gates en Azure Pipelines: Establecer compuertas de calidad automáticas en los flujos de CI/CD (Azure Pipelines / GitHub Actions) que ejecuten la totalidad de las 56 pruebas unitarias y la colección Newman de API en cada Pull Request. Se debe configurar el bloqueo obligatorio de cualquier integración si la cobertura de código desciende del 85% o si una sola aserción resulta fallida.

2. Pruebas de Carga y Concurrencia con k6 / JMeter: Diseñar e incorporar escenarios de prueba de estrés y concurrencia masiva utilizando herramientas como k6 o Apache JMeter, simulando ráfagas de 1,000 solicitudes simultáneas en periodos de alta demanda de citas para calibrar el comportamiento del pool de conexiones a la base de datos y la capacidad de autoescalabilidad del backend.

3. Monitoreo Sintético y Observabilidad en Producción: Implementar instrumentación distribuida mediante Azure Application Insights y OpenTelemetry para supervisar continuamente las métricas de telemetría en producción, detectando anomalías en tiempos de respuesta de consultas complejas y tasas de error de red antes de que impacten a los usuarios finales.

4. Gestión Centralizada de Secretos y Seguridad OWASP: Migrar la gestión de claves criptográficas y secretos de conexión hacia Azure Key Vault con rotación periódica automatizada de certificados, reforzando la seguridad de la API contra ataques descritos en OWASP Top 10 API Security mediante políticas de limitación de tasa (Rate Limiting) y validación de Content-Type.

5. Automatización Móvil en Granjas de Dispositivos Reales: Extender las pruebas de aceptación móvil a granjas de dispositivos físicos en la nube (tales como AWS Device Farm o BrowserStack) para certificar la compatibilidad del cliente .NET MAUI ante la fragmentación de fabricantes de Android (capas de personalización One UI, MIUI, MagicOS) y variaciones de densidad de pantalla.